

SISTEMA DE GESTIÓN “EDIFICIO MIRADOR”

(DAS) Documento Arquitectura de Software
Versión 1.0

Integrantes: Javiera Cid
Marjorie Portiño
Milton Saavedra

Identificación de Documento

Identificación	Informe de proyecto de software
Proyecto	Sistema de Gestión Edificio Mirador
Versión	1.0

Documento mantenido por	Equipo de desarrollo
Fecha de última revisión	03/04/2026
Fecha de próxima revisión	04/04/2026

Documento aprobado por	Profesor guía
Fecha de última aprobación	

Historia de Revisiones

Fecha	Versión	Descripción	Autor
02/04/2026	1.0	Creación inicio del documento	Marjorie Portiño
03/04/2026	1.0	Se añadieron definiciones, acrónimos y abreviaciones	Marjorie Portiño
04/04/2026	1.0	Inclusión de resumen ejecutivo y visión del sistema	Marjorie Portiño
06/04/2026	1.0	Se agregaron los requisitos funcionales.	Javiera Cid
08/04/2026	1.0	Se agregaron los requisitos no funcionales y se realizó una revisión general del informe y últimas correcciones.	Javiera Cid
08/04/2026	1.0	Se agregó la arquitectura del sistema.	Milton Saavedra

Tabla de Contenidos

1. INTRODUCCIÓN	4
1.1. CONTEXTO DEL PROBLEMA	4
1.2. PROPÓSITO	4
1.3. ÁMBITO	4
1.4. DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES	4
1.5. RESUMEN EJECUTIVO	4
1.6. ARQUITECTURA DEL SISTEMA	4
2. VISIÓN DEL SISTEMA	4
2.1. DESCRIPCIÓN GENERAL DEL SISTEMA	4
2.2. OBJETIVOS DEL SISTEMA	4
2.3. REQUERIMIENTOS FUNCIONALES	4
2.4. REQUERIMIENTOS NO FUNCIONALES	5
2.5. SUPUESTOS Y DEPENDENCIAS	5
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	5
3.2. JUSTIFICACIÓN DEL ESTILO SEGÚN EL CONTEXTO DEL SISTEMA	5
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS	5
4.1. VISTA DE ESCENARIO	5
4.1.1. <i>Propósito</i>	5
4.1.2. <i>Actores</i>	5
4.1.3. <i>Diagrama general de casos de uso</i>	5
4.1.4. <i>Diagrama de casos de uso específicos</i>	5
4.1.6. <i>Especificación de casos de uso</i>	5
4.2. VISTA LÓGICA	6
4.2.1. <i>Propósito</i>	6
4.2.2. <i>Diagrama de clases</i>	6
4.2.3. <i>Descripción diagrama de clases</i>	6
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	7
4.3.1. <i>Propósito</i>	7
4.3.2. <i>Diagrama de componente</i>	7
4.3.3. <i>Descripción diagrama de componente</i>	7
4.3.4. <i>Diagrama de paquete</i>	7
4.3.5. <i>Descripción diagrama de paquete</i>	7
4.4. VISTA DE PROCESOS	7
4.4.1. <i>PROPÓSITO</i>	7
4.4.2. <i>DIAGRAMA DE ACTIVIDAD</i>	7
4.4.3. <i>DESCRIPCIÓN DIAGRAMA DE ACTIVIDAD</i>	7
4.5. VISTA FÍSICA	7
4.5.1. <i>Propósito</i>	7
4.5.2. <i>Diagrama de despliegue</i>	7
4.5.3. <i>Descripción diagrama de despliegue</i>	7
5. REQUISITOS DE CALIDAD	7

5.1. PROPÓSITO	7
5.3. <i>Reglas y criterios de evaluación de calidad</i>	7
6. PRINCIPIOS DE DISEÑO APLICADOS	8
6.1. <i>Propósito</i>	8
6.2. PRINCIPIOS DE DISEÑO (POR EJEMPLO: ABSTRACCIÓN, ACOPLAMIENTO, COHESIÓN, ENCAPSULAMIENTO, MODULARIDAD)	8
7. PROTOTIPO	8
7.1. PROPÓSITO	8
7.2. MOCKUPS (IMÁGENES CON UNA BREVE DESCRIPCIÓN)	8
7.3. JUSTIFICAR HERRAMIENTAS DE PROTOTIPADO	8
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	8
8.1. PROPÓSITO	8
8.2. LISTA DE VERIFICACIÓN	8
8.3. ANÁLISIS Y MÉTRICAS DE RESULTADOS	8
9. CONTROL DE VERSIONES	8
9.1. PROPÓSITO	8
9.2. CONTROL DE VERSIÓN UTILIZADO (JUSTIFICAR EL TIPO DE CONTROL DE VERSIÓN UTILIZAD (FECHA, SEMÁNTICA O SECUENCIAL)	8
9.3. JUSTIFICAR HERRAMIENTAS DE VERSIONAMIENTO	8
7. CONCLUSIONES	8
8. BIBLIOGRAFÍA	8
9. ANEXOS	8
9.1. CARTA GANTT	8

INTRODUCCIÓN

1.1. Contexto del Problema

El edificio El Mirador, ubicado en una zona céntrica de la ciudad, enfrenta una serie de dificultades en la gestión en sus gastos comunes y en la administración de la comunidad. Con 160 departamentos y una alta rotación entre propietarios y arrendatarios, la administración debe coordinar múltiples intereses y necesidades.

Actualmente los procesos de cobro y registro de pagos se realizan de manera manual, lo que genera ineficiencias, errores y retrasos en la gestión.

La morosidad de los residentes afecta directamente el flujo de caja, provocando problemas de liquidez y retrasos en mantenciones esenciales. Además, la falta de un sistema automatizado dificulta la identificación de residentes morosos y reduce la transparencia en la comunicación entre la administración y la comunidad. Esta situación ha generado desconfianza, tensiones y aumento de los costos operativos, poniendo en riesgo la estabilidad financiera y la calidad de vida de los residentes.

1.2. Propósito

El propósito de este documento es definir los requisitos para el desarrollo del Sistema de Gestión de gastos comunes del edificio El Mirador, con el fin de optimizar los procesos administrativos y mejorar la transparencia en la relación con los residentes.

Está dirigido a la administración del edificio, los residentes (propietarios y arrendatarios) y el equipo de desarrollo de software, con el objetivo de establecer el alcance del sistema y garantizar que cumpla con las necesidades del negocio y de los usuarios.

1.3. Ámbito

El sistema estará enfocado en mejorar la gestión de los gastos comunes del edificio El Mirador, apoyando a la administración en tareas como el registro de residentes, control de pagos, envío de notificaciones, la identificación de deudas y la generación de información clara para todos los involucrados. Además, permitirá a los residentes acceder fácilmente a sus datos, revisar su estado de pago y mantenerse informados.

Este sistema será utilizado principalmente por el administrador del edificio y los residentes (propietarios y residentes), con el objetivo de mejorar la comunicación y la transparencia dentro de la comunidad.

Se centrará en la gestión interna del edificio y no incluirá el desarrollo de plataformas externas de pago, aunque sí podrá adaptarse a futuras integraciones.

1.4. Definiciones, acrónimos y abreviaciones

ACRÓNIMO	DESCRIPCIÓN
<i>SGC</i>	Sistema de Gestión de Gastos Comunes
<i>ER</i>	Especificación de requisitos
<i>Adm.</i>	Administrador del edificio
<i>UI</i>	Interfaz de Usuario
<i>Rep. Fin.</i>	Reportes Financiero
<i>WbApp</i>	Aplicación Web
<i>Not.</i>	Notificaciones

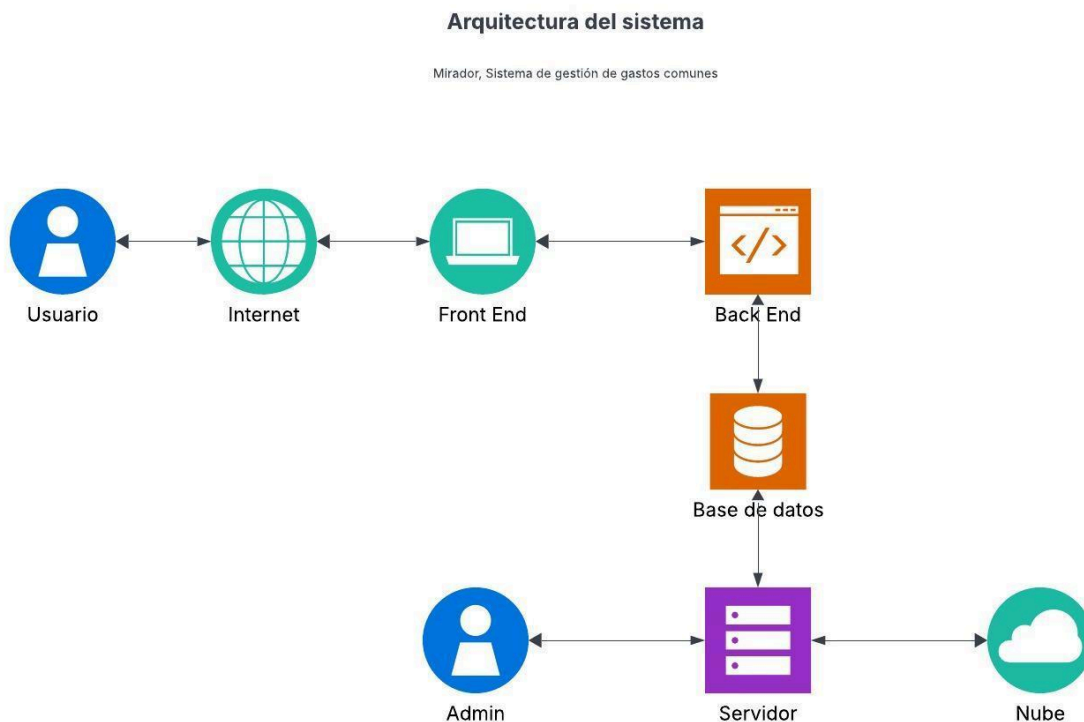
1.5. Resumen ejecutivo

El Sistema de gestión de gastos comunes del edificio El Mirador, busca optimizar los procesos administrativos, reducir la morosidad y mejorar la transparencia en la información.

Actualmente, la administración enfrenta problemas debido a procesos manuales, falta de control en los pagos y dificultades en la comunicación con los residentes. Esto genera retrasos en mantenciones, conflictos y una gestión poco eficiente.

La solución propuesta consiste en un sistema automatizado que permita gestionar residentes, registrar pagos, emitir notificaciones y generar reportes, facilitando la toma de decisiones y mejorando la convivencia dentro de la comunidad.

1.6. Arquitectura del sistema



2. VISIÓN DEL SISTEMA

2.1. Descripción general del sistema

El sistema será una plataforma web y móvil que permitirá a la administración y los residentes gestionar de manera eficiente los pagos, consultas y reportes asociados a la vida comunitaria del edificio El Mirador.

A través de esta solución, se podrán automatizar procesos como el registro de pagos, Control de morosidades y generación de informes, lo que ayudará a reducir errores y mejorar los tiempos de gestión.

Su propósito es optimizar los procesos administrativos, reducir la morosidad y mejorar la transparencia en la comunicación entre la administración y los residentes, contribuyendo a una gestión más ordenada y eficiente.

2.2. Objetivos del sistema

Desarrollar un sistema que permita gestionar de manera eficiente, transparente y accesible los gastos comunes del edificio El Mirador.

- Optimizar la gestión administrativa de los gastos comunes
- Automatizar el registro y control de pagos.
- Reducir la morosidad mediante notificaciones y reportes.
- Facilitar el pago de cuotas y servicios asociados.
- Mejorar la organización y acceso a la información en tiempo real.
- Aumentar la satisfacción de los residentes mediante una gestión confiable y accesible.
- Fortalecer la transparencia y la comunicación entre la administración y los residentes.
- Disminuir los costos operativos asociados a procesos manuales.

2.3. Requerimientos funcionales.

- Iniciar Sesión (Administración)
- Iniciar Sesión (Residentes)
- Iniciar Sesión (Personal)
- Registrar Propietario
- Registrar Arrendatario
- Registrar Personal
- Actualizar Datos de Emergencia
- Desactivar Residente
- Consultar Residente
- Calcular Cuota Gastos Comunes
- Registrar Gastos Extraordinarios
- Registrar Otros Servicios

- Visualizar Cobros en Tiempo Real
- Registrar Pago (Propietario)
- Registrar Pago (Arrendatario)
- Validar Pago
- Generar Recibos
- Identificar Morosos
- Enviar Recordatorios de Pago
- Emitir Notificaciones de Cobro
- Enviar Solicitud de Mantenimiento
- Enviar Solicitud de Reparación
- Enviar Solicitud de Servicios Generales
- Enviar Queja a Administración
- Gestionar Solicitud
- Gestionar Quejas
- Generar Informe de Residentes
- Generar Informe del Personal
- Generar Informe de Instalaciones
- Generar Informe de Pagos
- Generar Informe de Morosidades
- Consultas en Pantalla

2.4. Requerimientos no funcionales

- Interfaz Intuitiva (Administración)
- Interfaz Intuitiva (Residentes)
- Interfaz Simplificada (Personal)
- Actualización en Tiempo Real
- Velocidad de Carga Masiva
- Precisión en la Cobranza
- Mitigación de Errores de Ingreso
- Inmutabilidad de Recibos
- Privacidad Financiera
- Confidencialidad de Quejas
- Respaldo Anti-Pérdidas
- Trazabilidad de Acciones
- Ejecución Autónoma de Procesos
- Eficiencia en Notificaciones
- Acceso Permanente
- Soporte Multi-Usuario
- Formato de Informes
- Código Documentado

2.5. Supuestos y dependencias

Supuestos

- Los residentes cuentan con acceso a internet.
- La administración utilizará el sistema de forma continua.
- Los usuarios poseen conocimientos básicos de tecnología.
- La información ingresada será correcta y actualizada.

Dependencias

- Disponibilidad de internet para el uso del sistema.
- Infraestructura tecnológica (servidor o hosting).
- Posible integración con métodos de pagos externos.
- Capacitación inicial para los usuarios.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico adoptado (ej. monolítico, microservicios, SOA, capas)

3.2. Justificación del estilo según el contexto del sistema

3.3. Patrones de diseño aplicados (ej. patrón MVC, repositorio, etc.)

4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

4.1. VISTA DE ESCENARIO

4.1.1. Propósito

4.1.2. Actores

4.1.3. Diagrama general de casos de uso

4.1.4. Diagrama de casos de uso específicos

4.1.5. Lista de casos de uso

Código	Nombre	Actores
CU-001-001	Exportar saldos y puntos a vencer	
CU-002-001	Exportar actividades	
CU-002-002	Importar deuda vencida por PDV	
CU-004-001	Generación Archivo PDA Importación	

4.1.6. Especificación de casos de uso

Caso de Uso	[Nombre del Caso de Uso]	Identificador: [Del caso de uso]
Actores	[Listado de los actores que tienen participación en el caso de uso]	
Tipo	[Tipo de caso de uso, primario, secundario, opcional]	
Referencias	[Requerimientos o funcionalidades incluidas en este caso de uso. Casos de uso relacionados.]	
Precondición	[Condiciones sobre el estado del sistema que deben cumplirse para iniciar el caso de uso]	
Postcondición	[Efectos inmediatos que tienen la ejecución del caso de uso sobre el estado del sistema]	
Descripción	[Descripción del caso de uso]	
Resumen	[Resumen de alto nivel del funcionamiento]	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
[Nro. de paso]	[Actor ejecutor o especifica si es el sistema o subsistema]	[Descripción del paso actividad ejecutado]
[Se describe el proceso o secuencia de pasos ejecutadas usando frases cortas]		

[Cada paso del proceso puede ser ejecutado por los Actores o por el sistema] [Se describe la secuencia de acciones realizadas por los actores y la secuencia de actividades realizada por el sistema como respuesta].
--

CURSO ALTERNATIVO

Nro.	Descripción de acciones alternas
[Número de paso]	[Descripción de la secuencia de acciones alternas para el número de actividad indicado. Debe hacer referencia al número de paso en el curso normal]
[Cada paso descrito en el curso normal, puede tener actividades alternas, según la distribución de escenarios que ocurra en el flujo de procesos, en esta ficha se completa para cada actividad (haciendo referencia a su número) las posibles secuencias alternas]	

4.2. VISTA LÓGICA

4.2.1. Propósito

4.2.2. Diagrama de clases

4.2.3. Descripción diagrama de clases

4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO

- 4.3.1. Propósito
- 4.3.2. Diagrama de componente
- 4.3.3. Descripción diagrama de componente
- 4.3.4. Diagrama de paquete
- 4.3.5. Descripción diagrama de paquete

4.4. VISTA DE PROCESOS

- 4.4.1. Propósito
- 4.4.2. Diagrama de actividad
- 4.4.3. Descripción diagrama de actividad

4.5. VISTA FÍSICA

- 4.5.1. Propósito
- 4.5.2. Diagrama de despliegue
- 4.5.3. Descripción diagrama de despliegue

5. REQUISITOS DE CALIDAD

5.1. Propósito

5.2. Atributos de calidad (por ejemplo: Usabilidad, Accesibilidad (WCAG), Rendimiento, Mantenibilidad, Seguridad Portabilidad)

ATRIBUTO DE CALIDAD	DESCRIPCION	JUSTIFICACIÓN

5.3. Reglas y criterios de evaluación de calidad

Cómo se medirá el cumplimiento de cada atributo (ej. tiempo de carga < 2 seg, puntuación Nielsen de usabilidad, cumplimiento WCAG nivel AA, etc.).

Herramientas o métodos que se utilizarán(pruebas de carga, pruebas heurísticas, inspecciones, validación con usuarios, etc)

6. PRINCIPIOS DE DISEÑO APLICADOS

6.1. Propósito

6.2. Principios de diseño (por ejemplo: abstracción, acoplamiento, cohesión, encapsulamiento, modularidad)

PRINCIPIO	DESCRIPCIÓN	APLICACIÓN EN EL SISTEMA
Cohesión	Cada módulo o clase tiene una única responsabilidad bien definida.	Los servicios están diseñados para realizar tareas específicas y no múltiples funciones

7. PROTOTIPO

7.1. Propósito

7.2. Mockups (imágenes con una breve descripción)

7.3. Justificar herramientas de prototipado

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

8.1. Propósito

8.2. Lista de verificación

8.3. Análisis y métricas de resultados

9. CONTROL DE VERSIONES

9.1. Propósito

9.2. Control de versión utilizado (justificar el tipo de control de versión utilizado (fecha, semántica o secuencial)

9.3. Justificar herramientas de versionamiento

7. CONCLUSIONES

8. BIBLIOGRAFÍA

9. ANEXOS

9.1. Carta Gantt

[illegible]